47. Design Payment Gateway

Key Terms:

Term	Full Name	What it does
PCI-DSS	Payment Card Industry Data Security Standard	Security requirements for card processing
TPS	Transactions Per Second	Throughput measure for payment systems
3DS	3D Secure	Additional authentication (Verified by Visa, etc.)
Tokenization	Card data replacement	Replace card number with non-sensitive token
Idempotency Key	Duplicate prevention	Ensures same request isn't processed twice
Chargeback	Transaction dispute	Customer disputes charge with bank
ACH	Automated Clearing House	US bank-to-bank transfer system
Settlement	Funds transfer	Moving money from buyer to seller account
Acquirer	Merchant's bank	Bank that receives payment for merchant
Issuer	Cardholder's bank	Bank that issued the credit card

Problem Statement

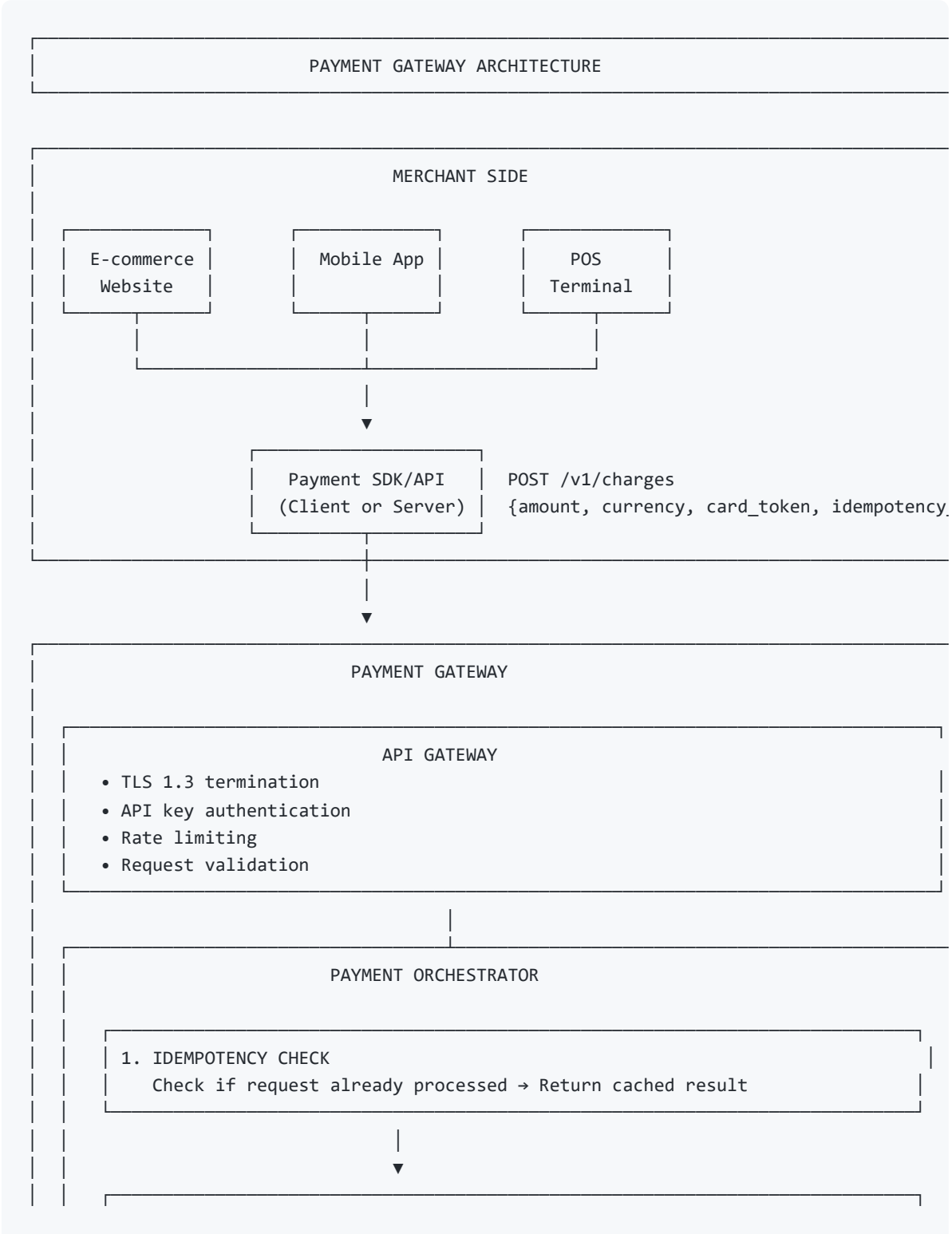
Design a payment gateway like Stripe or PayPal that securely processes credit card transactions with fraud detection, PCI compliance, and high availability.

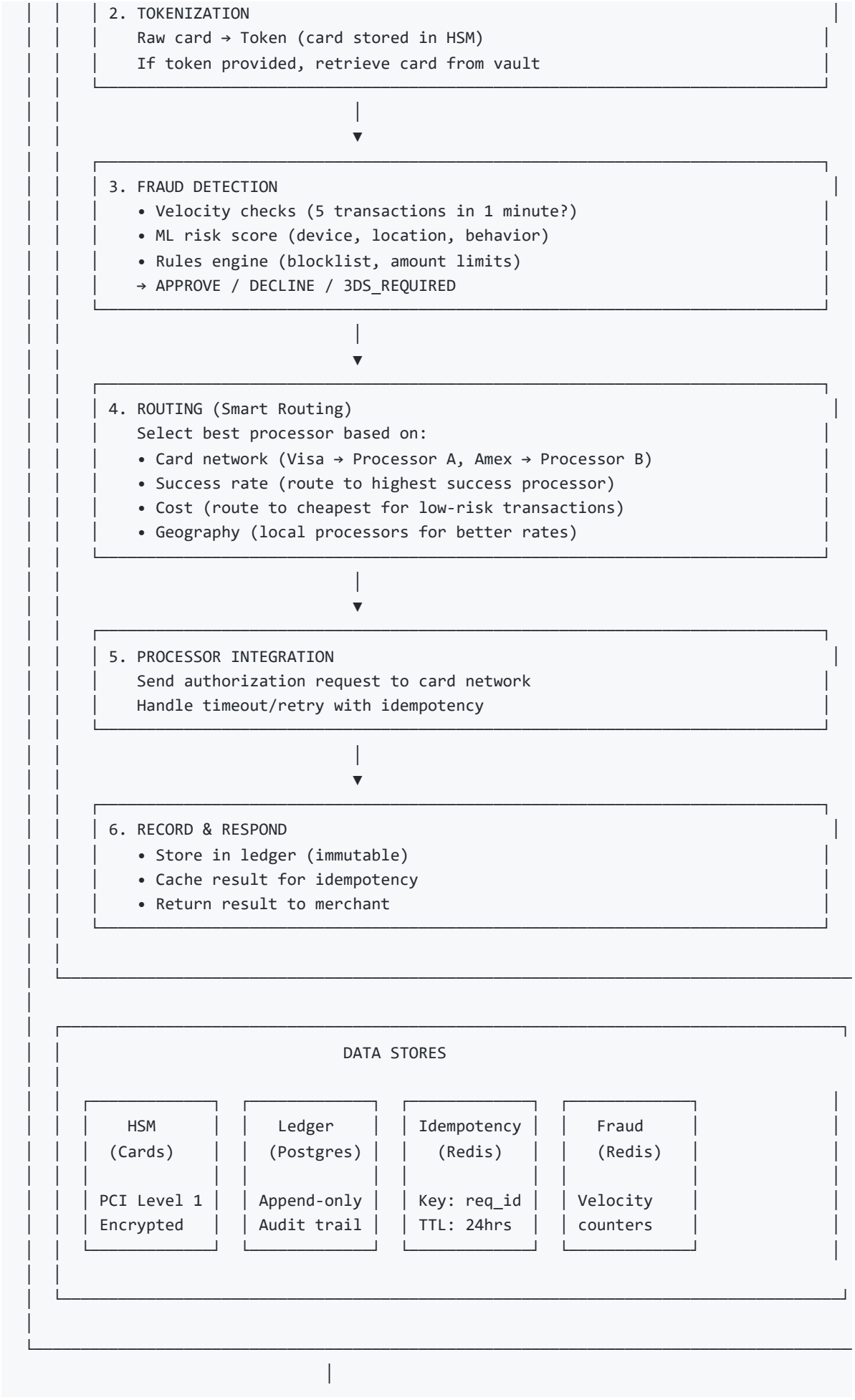
Clarifying Questions to Ask

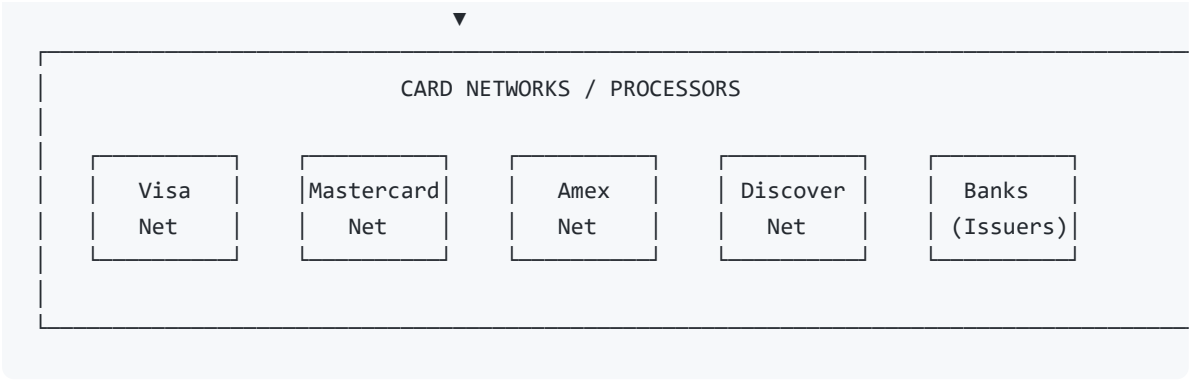
1. **Scale**: Transactions per second? (1K-100K TPS?)
2. **Payment types**: Credit cards only, or also ACH, wallets, crypto?

3. **Regions**: Global or specific markets? Multi-currency?
4. **Compliance**: PCI-DSS level? (Level 1 for >6M transactions/year)
5. **Settlement**: Real-time or batch settlement?
6. **Retry policy**: How to handle network failures?
7. **Chargebacks**: Dispute handling workflow?

High-Level Design




2. TOKENIZATION
Raw card → Token (card stored in HSM)
If token provided, retrieve card from vault
3. FRAUD DETECTION
• Velocity checks (5 transactions in 1 minute?)
• ML risk score (device, location, behavior)
• Rules engine (blocklist, amount limits)
→ APPROVE / DECLINE / 3DS_REQUIRED
4. ROUTING (Smart Routing)
Select best processor based on:
• Card network (Visa → Processor A, Amex → Processor B)
• Success rate (route to highest success processor)
• Cost (route to cheapest for low-risk transactions)
• Geography (local processors for better rates)
5. PROCESSOR INTEGRATION
Send authorization request to card network
Handle timeout/retry with idempotency
6. RECORD & RESPOND
• Store in ledger (immutable)
• Cache result for idempotency
• Return result to merchant
DATA STORES
HSM
(Cards)
PCI Level 1
Encrypted
Ledger
(Postgres)
Append-only
Audit trail
Idempotency
(Redis)
Key: req_id
TTL: 24hrs
Fraud
(Redis)
Velocity
counters



API Design

Payment APIs

Method	Endpoint	Description
`POST`	`/api/v1/payments`	Create a payment
`GET`	`/api/v1/payments/{paymentId}`	Get payment status
`POST`	`/api/v1/payments/{paymentId}/capture`	Capture authorized payment
`POST`	`/api/v1/payments/{paymentId}/cancel`	Cancel/void payment
`POST`	`/api/v1/refunds`	Create a refund
`GET`	`/api/v1/refunds/{refundId}`	Get refund status

Create Payment Request

```
POST /api/v1/payments
{
  "amount": 9999,
  "currency": "USD",
  "payment_method": {
    "type": "card",
    "token": "tok_abc123"     // tokenized card
  },
  "merchant_id": "merch_456",
  "order_id": "order_789",
  "capture": false,            // auth-only (capture later)
  "idempotency_key": "idem_001",
  "metadata": { "order_type": "subscription" }
}

Response: 201 Created
{
  "payment_id": "pay_xyz",
```

```
    "status": "authorized",
    "amount": 9999,
    "currency": "USD",
    "created_at": "2025-03-15T10:00:00Z"
  }
```

Webhook APIs

Method	Endpoint	Description
`POST`	`/api/v1/webhooks`	Register webhook endpoint
`GET`	`/api/v1/webhooks/{webhookId}/deliveries`	Get delivery history

Webhook Events

Event	Payload
`payment.authorized`	`{ paymentId, amount, status }`
`payment.captured`	`{ paymentId, capturedAmount }`
`payment.failed`	`{ paymentId, error, failureCode }`
`refund.completed`	`{ refundId, paymentId, amount }`

Idempotency (Critical for Payments)

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                          IDEMPOTENCY IMPLEMENTATION                         │
└─────────────────────────────────────────────────────────────────────────────┘

PROBLEM:
  Client sends: POST /charges {amount: $100, idempotency_key: "abc123"}
  Network timeout - did it succeed?
  Client retries - will user be charged twice?

SOLUTION:
┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│   REQUEST FLOW:                                                             │
│                                                                             │
│   1. Receive request with idempotency_key = "abc123"                        │
│                                                                             │
│   2. Check Redis: GET idempotency:abc123                                    │
│      ┌──────────────────────────────────────────────────────────────────┐   │
│      │ a) KEY EXISTS → Return cached response (charge NOT processed again)  │
│      │ b) KEY NOT EXISTS → Continue to step 3                           │   │
```

```
│       └──────────────────────────────────────────────────────────────────────┘
│
│   3. SET idempotency:abc123 "processing" EX 86400 NX
│       ┌──────────────────────────────────────────────────────────────────────┐
│       │ NX = Set only if not exists (atomic)                                 │
│       │ If SET fails → Another request is processing, wait or return conflict│
│       └──────────────────────────────────────────────────────────────────────┘
│
│   4. Process payment
│
│   5. Store result: SET idempotency:abc123 {result_json} EX 86400
│
│   6. Return result
│
└──────────────────────────────────────────────────────────────────────────────┘

IDEMPOTENCY KEY STORAGE:
{
  "key": "abc123",
  "status": "completed",
  "result": {
    "charge_id": "ch_xyz789",
    "amount": 10000,
    "currency": "usd",
    "status": "succeeded"
  },
  "created_at": "2024-01-15T10:30:00Z",
  "expires_at": "2024-01-16T10:30:00Z"
}
```

Tokenization & PCI Compliance

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          TOKENIZATION FLOW                                   │
└──────────────────────────────────────────────────────────────────────────────┘

INITIAL CARD SUBMISSION (Client-side):
┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│   Browser/App                        Tokenization Service (PCI Scope)        │
│        │                                        │                           │
│        │  POST /tokens                          │                           │
│        │  {card: "4242424242424242",            │                           │
│        │   exp: "12/25", cvc: "123"}            │                           │
│        │──────────────────────────────────────►│                           │
│        │                                        │                           │
│        │                                ┌───────┴──────┐                    │
│        │                                │ Encrypt card │                    │
│        │                                │ Store in HSM │                    │
│        │                                │ Generate token│                   │
│        │                                └───────┬──────┘                    │
│        │                                        │                           │
│        │  {token: "tok_abc123"}                 │                           │
```

```
│        │◀──────────────────────────────────────┐
│        │                                       │
│
│   TOKEN USAGE (Server-side - OUT of PCI scope):
│   POST /charges
│   {amount: 10000, source: "tok_abc123"}
│
│   Merchant server NEVER sees actual card number!
│
└──────────────────────────────────────────────────────────────────────────────────────────────────
```

PCI-DSS REQUIREMENTS:

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│
│   DO:
│     ✓ Encrypt card data at rest (AES-256)
│     ✓ Encrypt in transit (TLS 1.3)
│     ✓ Store encryption keys in HSM
│     ✓ Tokenize before storing
│     ✓ Mask card numbers in logs (****4242)
│     ✓ Quarterly vulnerability scans
│     ✓ Annual penetration testing
│
│   DON'T:
│     ✗ Log full card numbers
│     ✗ Store CVV/CVC (ever!)
│     ✗ Store unencrypted card data
│     ✗ Email card numbers
│
└──────────────────────────────────────────────────────────────────────────────────────────────────
```

Fraud Detection

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│                              FRAUD DETECTION LAYERS
└──────────────────────────────────────────────────────────────────────────────────────────────────
```

LAYER 1: RULES ENGINE (Fast, <5ms)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│
│   Rule Examples:
│     • Amount > $10,000 → FLAG
│     • Country mismatch (card country ≠ IP country) → FLAG
│     • Card on blocklist → DECLINE
│     • New account + high value → FLAG
│
└──────────────────────────────────────────────────────────────────────────────────────────────────
```

LAYER 2: VELOCITY CHECKS (Redis counters)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────
│
│   Redis Keys (with TTL):
│     velocity:card:{hash}:1h   → Count transactions in last hour
```

```
│     velocity:ip:{ip}:1h       → Count from same IP
│     velocity:device:{id}:1d   → Count from same device
│
│   Thresholds:
│     > 5 transactions per card per hour → FLAG
│     > 10 transactions per IP per hour → FLAG
│     > 3 declines in a row → BLOCK
│
└──────────────────────────────────────────────────────────────────────────────

LAYER 3: ML MODEL (Risk scoring)
┌──────────────────────────────────────────────────────────────────────────────
│
│   Features:
│     • Transaction: amount, currency, merchant category
│     • Card: issuer, country, age, historical fraud rate
│     • Device: fingerprint, known device?, proxy/VPN?
│     • Behavior: time since last transaction, typical spending pattern
│     • Context: time of day, merchant reputation
│
│   Output: Risk score 0-100
│     0-30:  APPROVE
│     30-70: 3D SECURE (step-up authentication)
│     70-100: DECLINE
│
└──────────────────────────────────────────────────────────────────────────────
```

Ledger (Double-Entry Bookkeeping)

```sql
-- Immutable transaction ledger (append-only)
CREATE TABLE ledger_entries (
    id BIGINT PRIMARY KEY,
    transaction_id VARCHAR(50),
    account_id VARCHAR(50),      -- "merchant:123", "fees", "reserves"
    entry_type VARCHAR(10),      -- 'debit' or 'credit'
    amount DECIMAL(15, 2),
    currency VARCHAR(3),
    created_at TIMESTAMP,
    metadata JSONB
);

-- Every transaction creates balanced entries (debits = credits)
-- Example: $100 charge with 2.9% + $0.30 fee

-- 1. Charge customer card
INSERT INTO ledger_entries VALUES (1, 'txn_123', 'customer_card', 'debit', 100.00, '
INSERT INTO ledger_entries VALUES (2, 'txn_123', 'merchant:456', 'credit', 97.00, 'U
INSERT INTO ledger_entries VALUES (3, 'txn_123', 'stripe_fees', 'credit', 2.90, 'USD
INSERT INTO ledger_entries VALUES (4, 'txn_123', 'stripe_fixed_fee', 'credit', 0.10,
```

```
-- Verify: SUM(debits) = SUM(credits)
-- $100 = $97 + $2.90 + $0.10 ✓
```

Key Components

Component	Purpose	Technology
API Gateway	Auth, rate limiting	Kong / Azure APIM
Payment Orchestrator	Workflow management	Go / Java
Tokenization Vault	Secure card storage	HSM (Thales/AWS CloudHSM)
Fraud Service	Risk scoring	Python ML + Rules engine
Processor Gateway	Network integration	ISO 8583 adapters
Ledger	Audit trail	PostgreSQL (append-only)
Idempotency Store	Deduplication	Redis (24hr TTL)
Notification Service	Webhooks	Kafka + Workers

Scalability Considerations

- **Stateless services**: Any server can handle any request
- **Database sharding**: Shard ledger by merchant_id
- **Read replicas**: Reporting queries on replicas
- **Async processing**: Webhooks, settlement via queues
- **Circuit breakers**: Fail fast if processor is down
- **Multi-region**: Active-passive for DR, active-active for global

Trade-offs

Decision	Option A	Option B	Recommendation
Card storage	Own vault	Third-party (Stripe)	**Third-party** unless at scale
Idempotency	Database	Redis	**Redis** - faster, TTL built-in
Ledger	Single DB	Event sourcing	**Event sourcing** for audit
Fraud	Rules only	Rules + ML	**Both** - rules fast, ML accurate
3DS	Always	Risk-based	**Risk-based** - better UX

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
PostgreSQL	Double-entry ledger for immutable transaction audit trail	Tech Guide
HSM	Hardware Security Module for secure card tokenization and encryption	Tech Guide
Saga Pattern	Payment orchestration with compensations for failed/refunded transactions	Tech Guide
Redis	Idempotency store (24hr TTL), velocity counters for fraud detection	Tech Guide

How to Explain It Well

> **Opening**: "Payment gateways need to ensure exactly-once processing, PCI compliance, and fraud prevention. I'll design with idempotency keys, tokenization, and layered fraud detection."

> **Key talking points**:
>
> - "Idempotency keys ensure charge-once: store result, return cached on retry"
> - "Tokenization: raw card numbers never touch merchant servers - PCI scope reduced"
> - "Cards encrypted in HSM, only tokenization service can decrypt"
> - "Fraud: rules for fast checks, velocity counters, ML for complex patterns"
> - "Double-entry ledger: every transaction balances, immutable audit trail"
> - "Smart routing: select processor by card network, success rate, cost"

Common Follow-up Questions

1. **What if the processor times out?**
    - Don't retry blindly (might double-charge)
    - Query processor for transaction status
    - Use idempotency key for safe retry
2. **How do you handle chargebacks?**
    - Webhook from processor triggers dispute workflow

- Gather evidence (receipt, delivery confirmation)
- Submit to processor within deadline
- Reserve funds during dispute

3. **How do you ensure high availability?**

- Multi-region deployment
- Multiple processor integrations (failover)
- Circuit breakers to fail fast
- 99.99% SLA = <52 minutes downtime/year